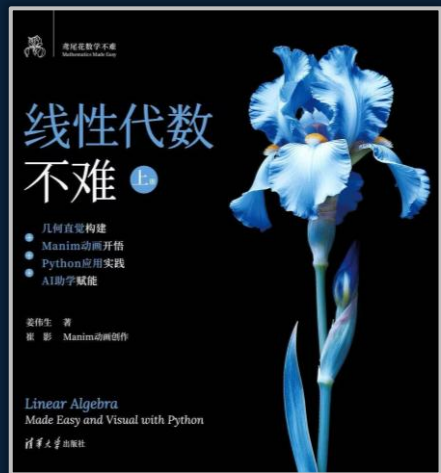


08 几何变换



8.1 几何变换和矩阵乘法

缩放、旋转、剪切、正交投影、镜像：从矩阵乘法视角回顾五种平面几何变换

鸢尾花书·《线性代数不难》 | 开源代码与书稿：<https://github.com/Visualize-ML>

本节你将掌握的核心技能

1

线性映射：通过矩阵乘法 $Ax=y$ 理解如何将输入向量 x 映射到输出向量 y

2

在线性变换下，单位正方形网格变为平行四边形网格：平行且等距，原点位置保持不变

3

特殊列向量的几何意义：零向量 0 、单位向量 e_1 、 e_2 、全1列向量 1

4

回顾矩阵连乘转置： $(AB)^T = B^T A^T$ ，理解连续几何变换的复合顺序

5

缩放：对角方阵；旋转：正交矩阵，行列式为1

6

剪切：上/下三角矩阵，主对角线元素均为1，行列式为1

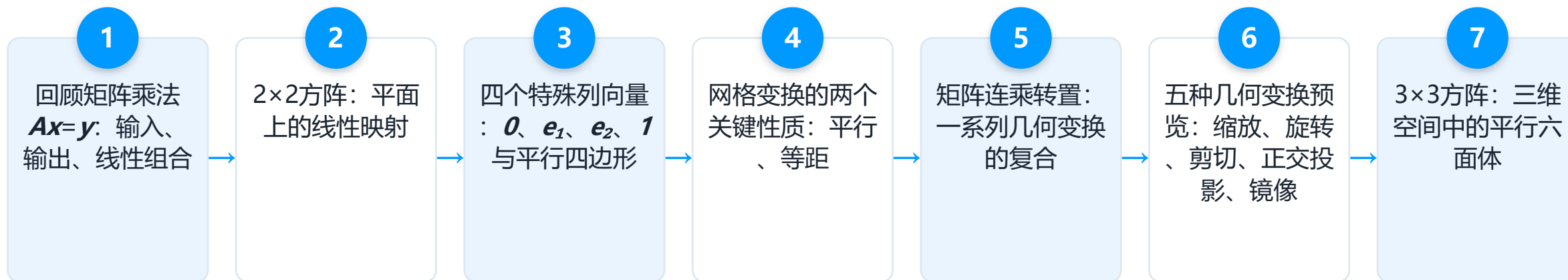
7

正交投影：降维，行列式为0，不可逆矩阵

8

镜像：正交矩阵，行列式为-1

从矩阵乘法出发，预览五种几何变换



本章每个几何变换，建议从这几方面看

- ① 矩阵乘法几何视角：矩阵对零向量 $\mathbf{0}$ 、单位向量 $\mathbf{e}_1$ 、 $\mathbf{e}_2$ 、全1列向量 $\mathbf{1}$ 的变换
- ② 看行列式：绝对值是否大于1；正负还是零；特殊矩阵（对角、上/下三角）的行列式
- ③ 从行列式看几何变换导致的面积或体积变换
- ④ 从逆矩阵（先看是否存在）看逆变换；特殊矩阵的逆矩阵
- ⑤ 线性组合角度看平行四边形网格、平行六面体形状变换，是否发生降维
- ⑥ 从基底角度看变换矩阵是否构成基底，以及构成怎样的基底
- ⑦ 变换矩阵的特点：是否对称、对角、正交、可逆，矩阵乘法是否满足交换律

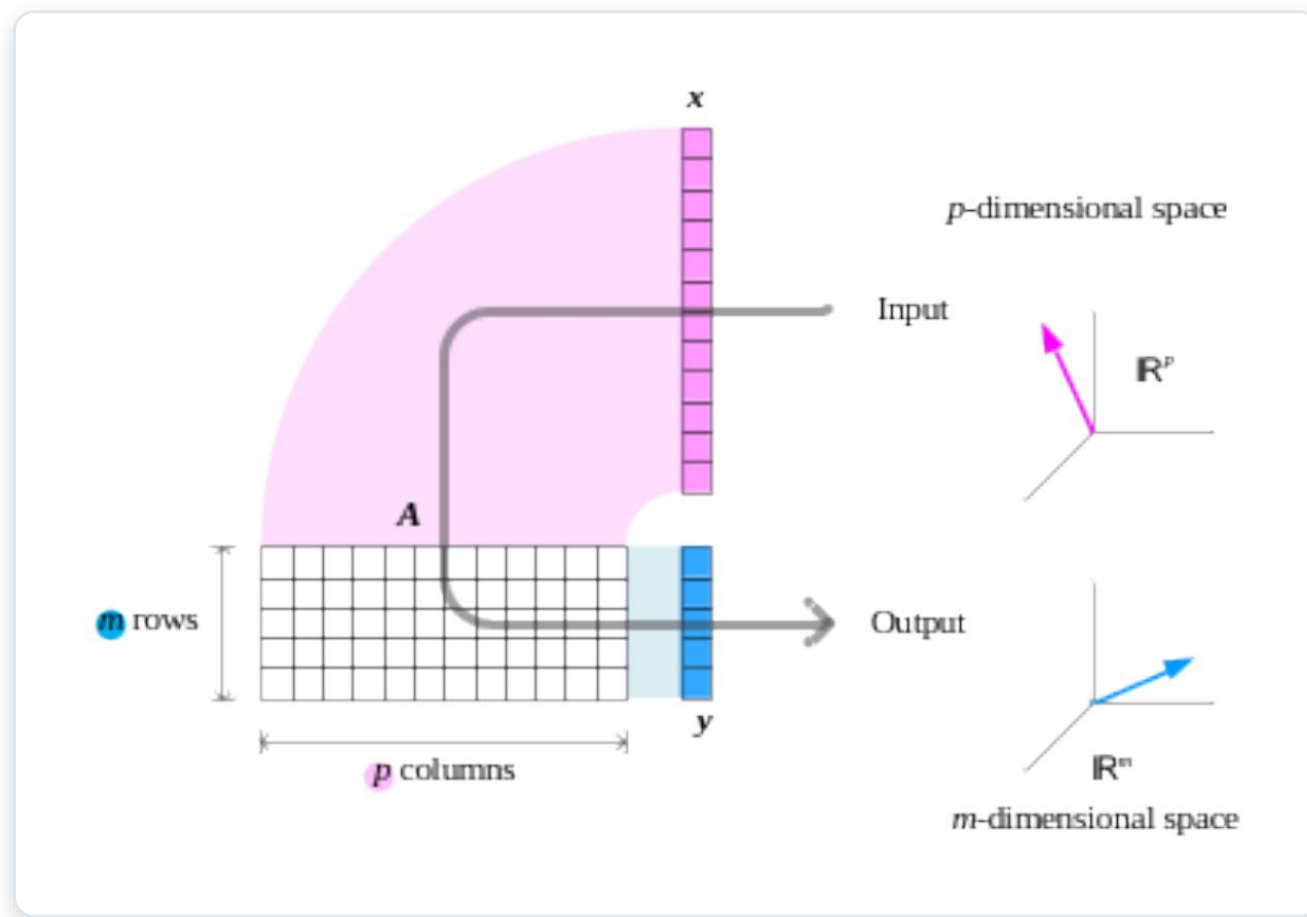
矩阵乘法 $Ax=y$: 输入与输出

$$Ax = y$$

- 矩阵乘法相当于用矩阵 A 对列向量 x 进行线性映射(linear mapping): p 维输入 x 映射为 m 维输出 y
- $(m \times p) @ (p \times 1)$ 中间的 p 被“消去”, 结果形状为 $(m \times 1)$; 从线性组合看, y 是 A 列向量的线性组合

图1 • 矩阵乘法 $Ax=y$ 的输入和输出

- 注意，图中矩阵 A 是扁平矩阵，其列向量不构成基底



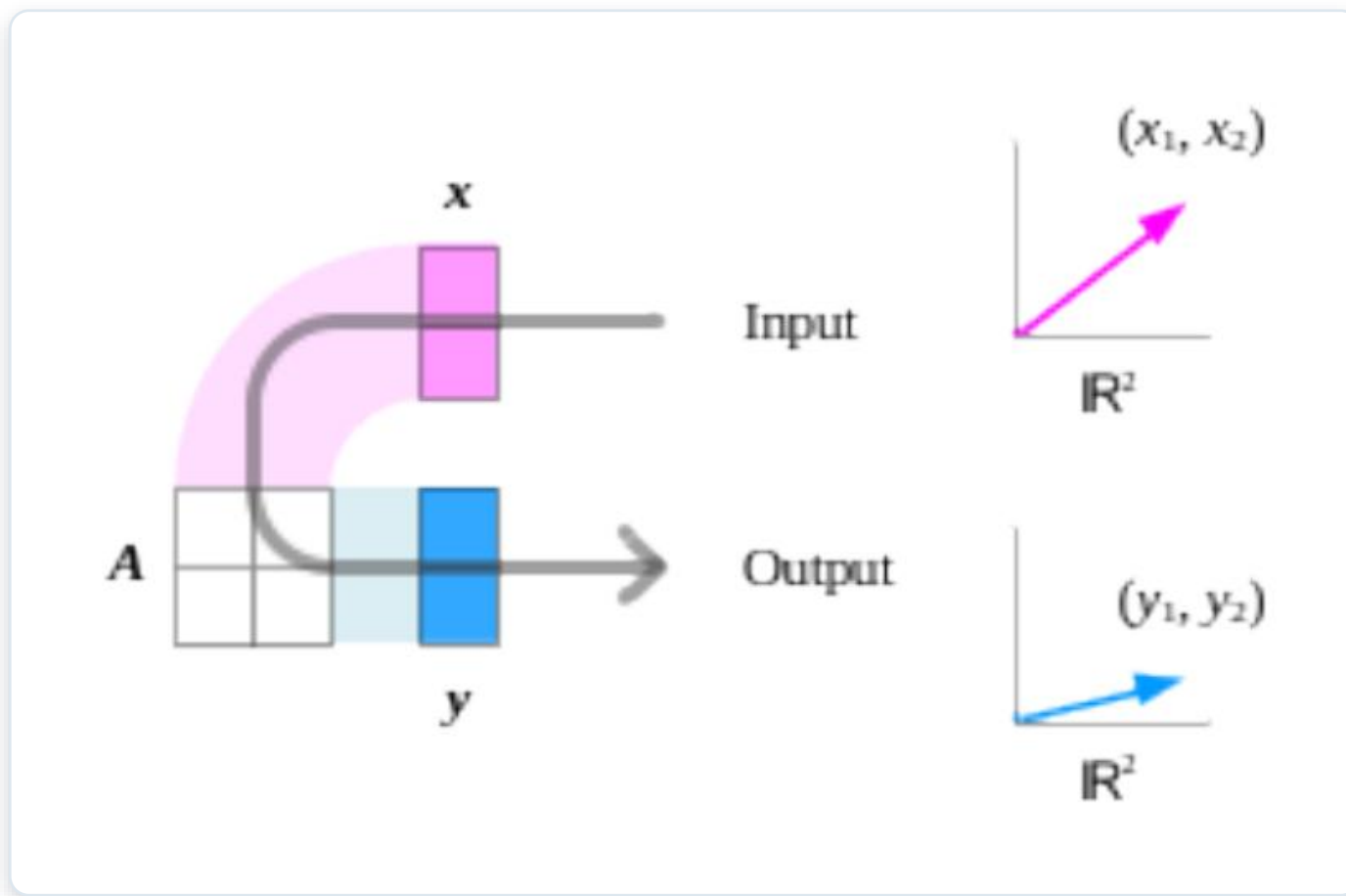
用矩阵乘法第三视角展开 $Ax=y$

$$Ax = x_1 a_1 + x_2 a_2 = y$$

- 输入 x 是 2×1 向量，对应平面上一点 (x_1, x_2) ；输出 y 也是 2×1 向量，对应点 (y_1, y_2)
- 列向量 y 是 a_1 、 a_2 的线性组合——这就是矩阵乘法的“第三视角”

图2 • $Ax=y$, 矩阵 A 为2×2

- 输入向量 x 、输出向量 y 都可以看作平面直角坐标系中起点位于原点的向量



四个特殊列向量的矩阵乘法

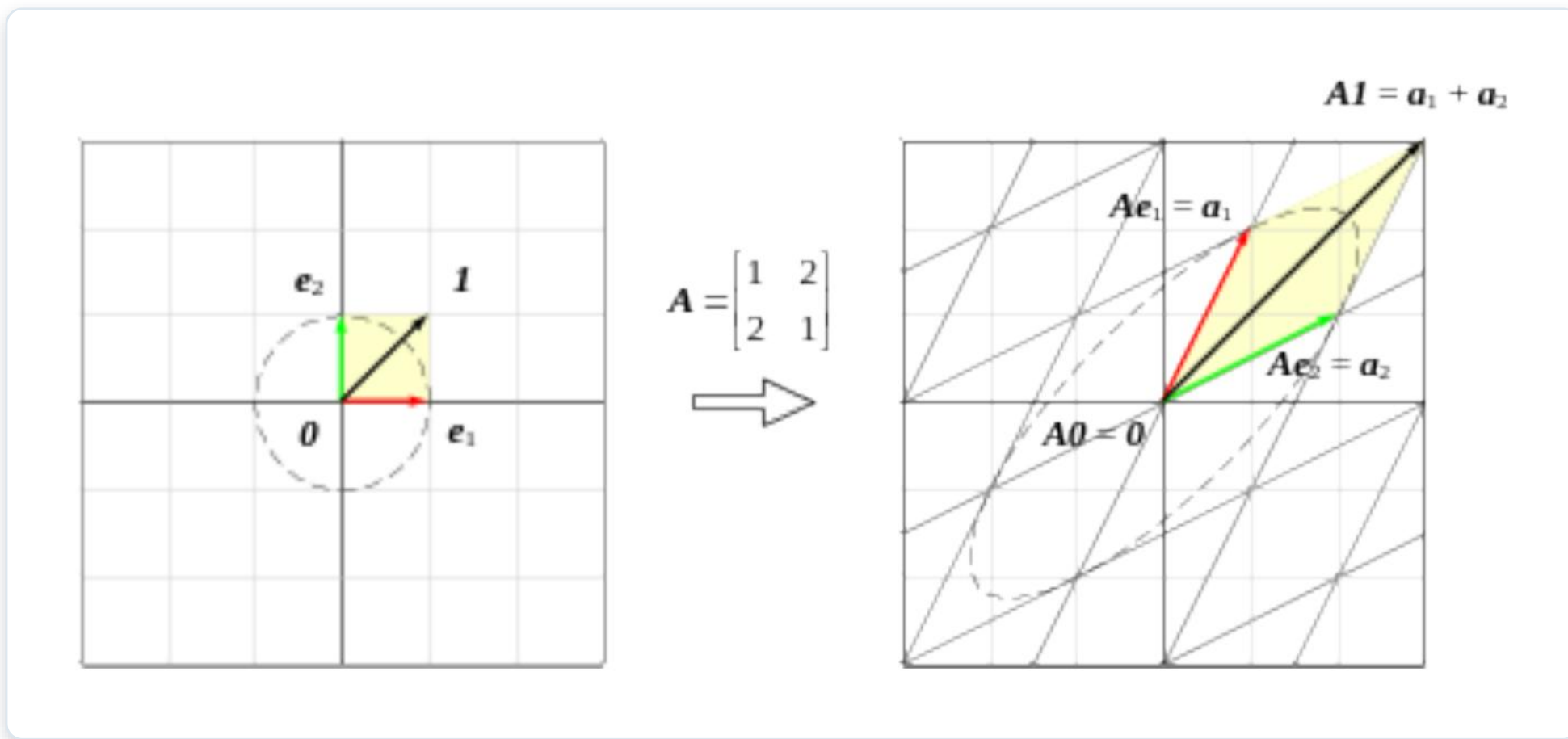
$$A\mathbf{0}=\mathbf{0}, \quad A\mathbf{e}_1=\mathbf{a}_1, \quad A\mathbf{e}_2=\mathbf{a}_2, \quad A\mathbf{1}=\mathbf{a}_1+\mathbf{a}_2$$

$$A\mathbf{0}=\mathbf{0} \quad A\mathbf{e}_1=\mathbf{a}_1 \quad A\mathbf{e}_2=\mathbf{a}_2 \quad A\mathbf{1}=\mathbf{a}_1+\mathbf{a}_2$$

- 零向量 $\mathbf{0}$ 经过线性变换，结果仍是 $\mathbf{0}$ ，即原点位置不变——这对线性变换特别重要
- $A\mathbf{e}_1$ 、 $A\mathbf{e}_2$ 分别提取矩阵 A 的第一、第二列向量； $A\mathbf{1}$ 是 A 两个列向量之和

图3 • 平面上四个特殊列向量

- 单位正方形（淡黄色）变为平行四边形； $\det(\mathbf{A}) = -3$ ，面积为3，负号说明180度逆时针看 $\mathbf{a}_1$ 落后 $\mathbf{a}_2$



网格变为平行四边形网格：平行且等距，原点不变

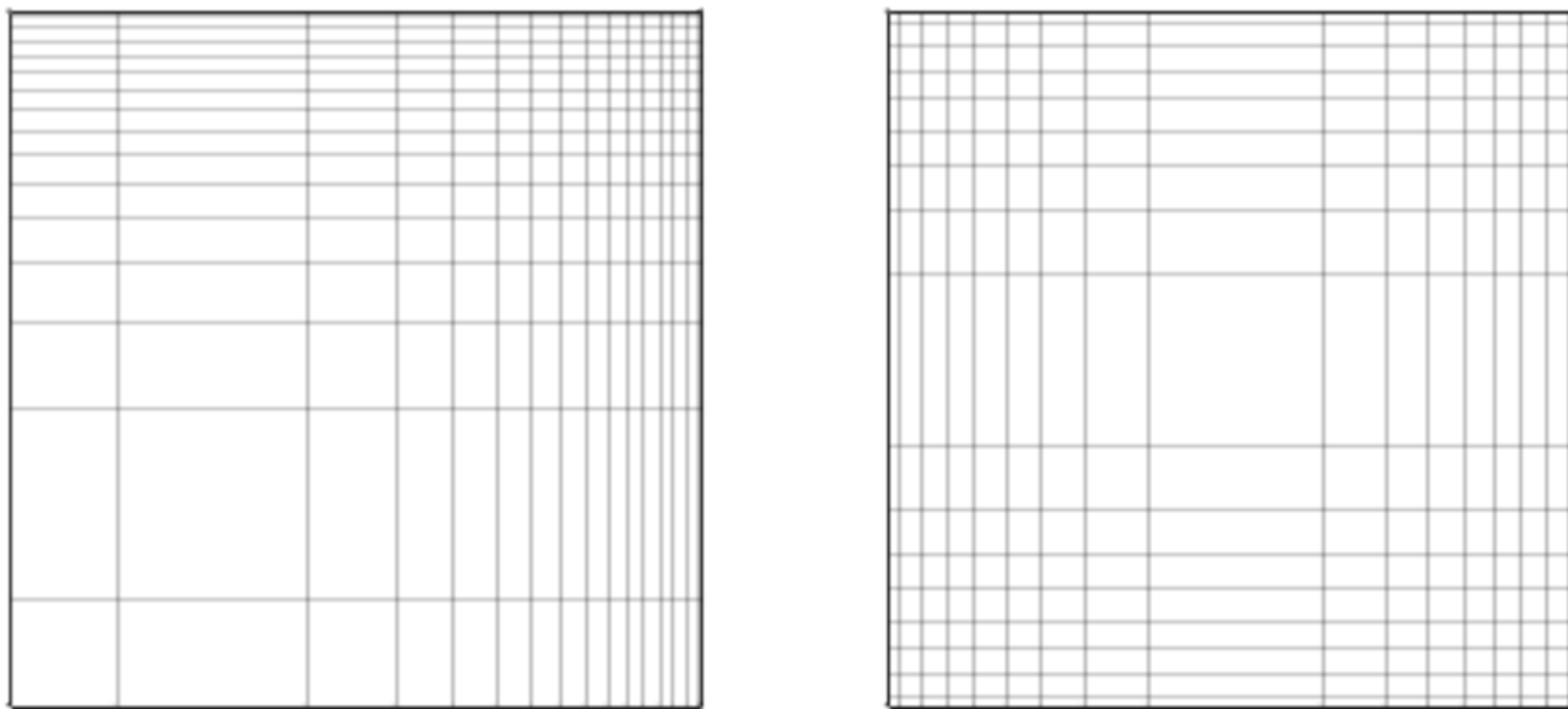
$$k_1 \mathbf{e}_1 + k_2 \mathbf{e}_2 \rightarrow k_1 \mathbf{a}_1 + k_2 \mathbf{a}_2$$

- 变换前，单位正方形网格对应标准正交基 $[\mathbf{e}_1, \mathbf{e}_2]$ 张成的空间；变换后若 $\mathbf{a}_1$ 、 $\mathbf{a}_2$ 线性无关，网格对应基底 $[\mathbf{a}_1, \mathbf{a}_2]$ 张成的空间
- 仅“平行”还不够，还必须“等距”——这才是判断是否为线性变换的关键标准

反例

图4 • 非线性变换

- 两个变换结果的网格都保持平行，但不再等距——因此图中两个变换都不是线性变换



矩阵连乘转置与几何变换的复合顺序

$$(Ax)^T = x^T A^T; \quad DCBAx$$

- 转置后用行向量描述同一个点； $(AB)^T = B^T A^T$ 说明连乘转置时，各矩阵顺序要反转
- 矩阵乘法 $DCBAx$ 表示一系列连续几何变换，对 x 的作用次序从右向左： $A \rightarrow B \rightarrow C \rightarrow D$

缩放、旋转、剪切、正交投影、镜像

缩放 按比例拉伸、压缩平面图形，变换矩阵为对角方阵

旋转 保持长度和角度不变，变换矩阵为正交矩阵，行列式为1

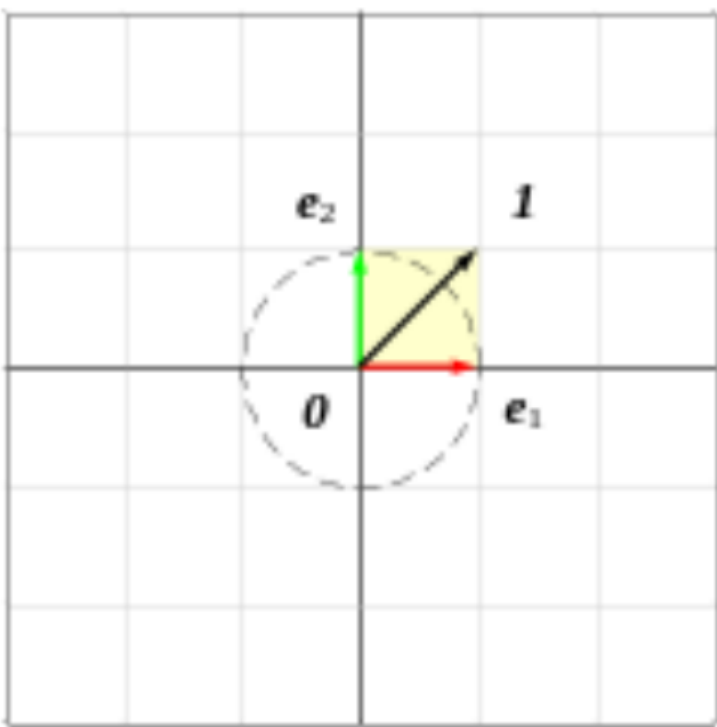
剪切 图形沿某方向倾斜变形，变换矩阵为上/下三角矩阵（对角元均为1），行列式为1

正交投影 将图形垂直投影到某子空间，发生降维，行列式为0，矩阵不可逆

镜像 关于过原点的直线对称翻转，变换矩阵为正交矩阵，行列式为-1

图5 · 矩阵 A 为 2×2 ，缩放

- 单位正方形网格变为正方形/矩形网格； $\mathbf{a}_1$ 、 $\mathbf{a}_2$ 线性无关时，基底 $[\mathbf{a}_1, \mathbf{a}_2]$ 为正交基



$$A = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$$

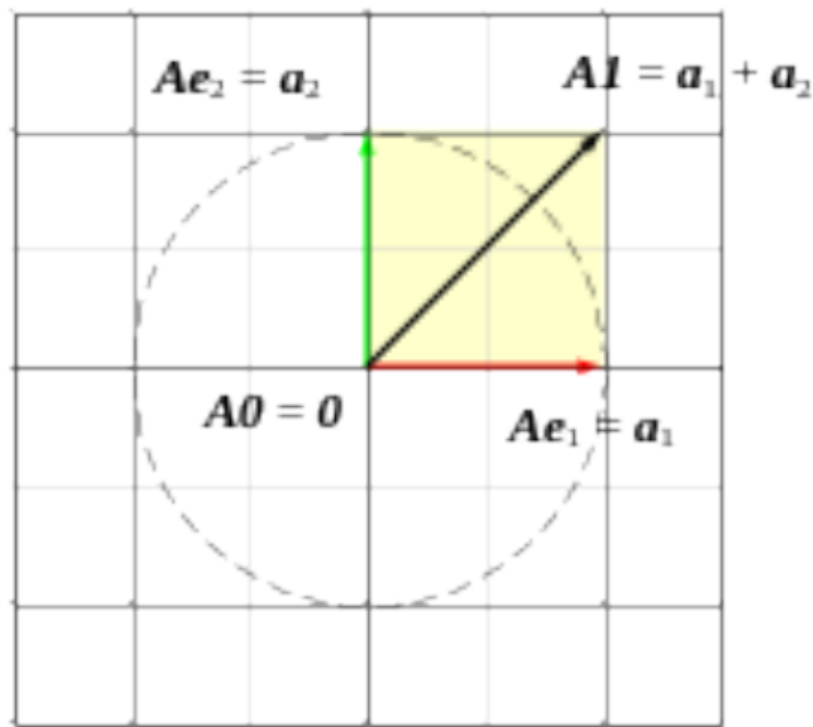


图6 · 矩阵 A 为 2×2 , (绕原点)旋转

- 横平竖直的单位正方形网格 (标准正交基) 变为旋转后的单位正方形网格 (规范正交基); A 满足 $AA^T = A^T A = I$

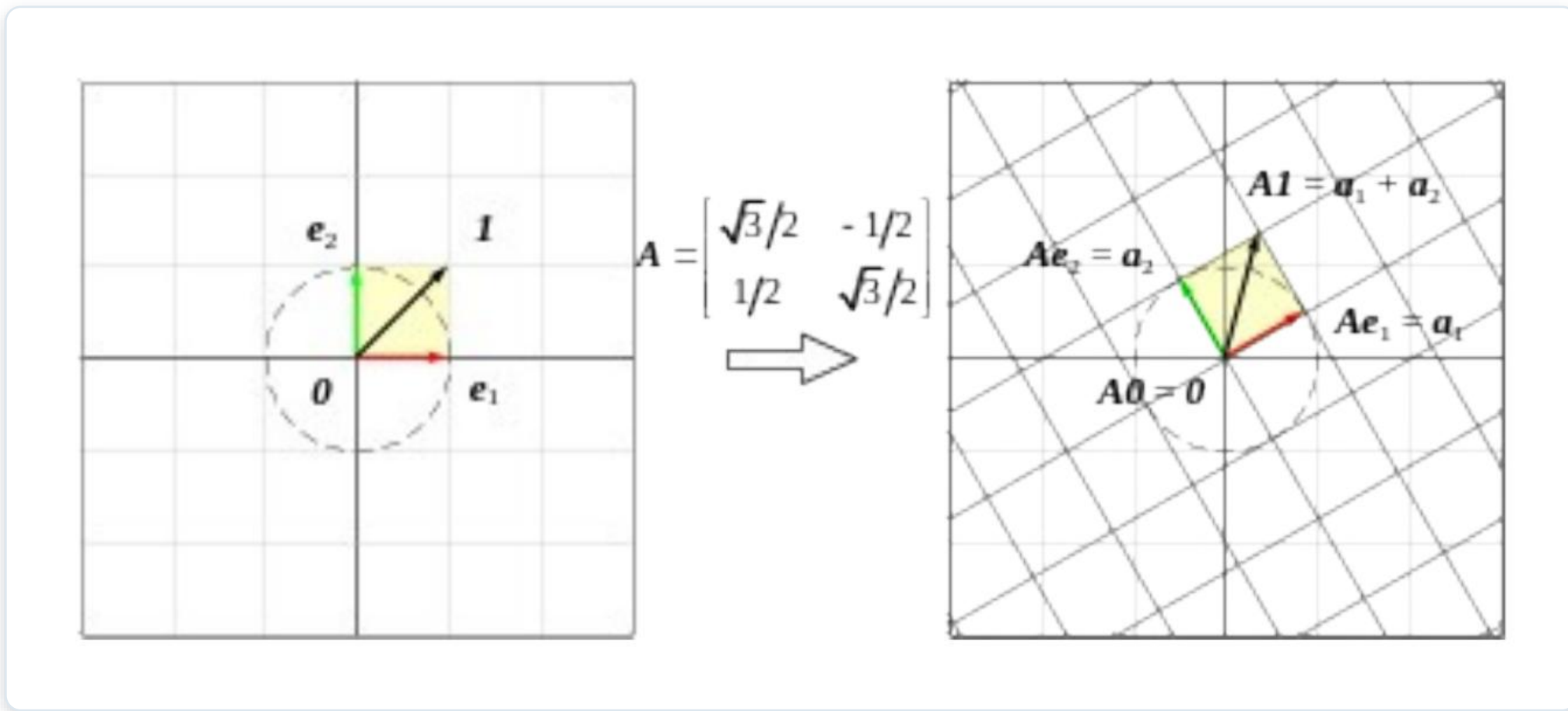
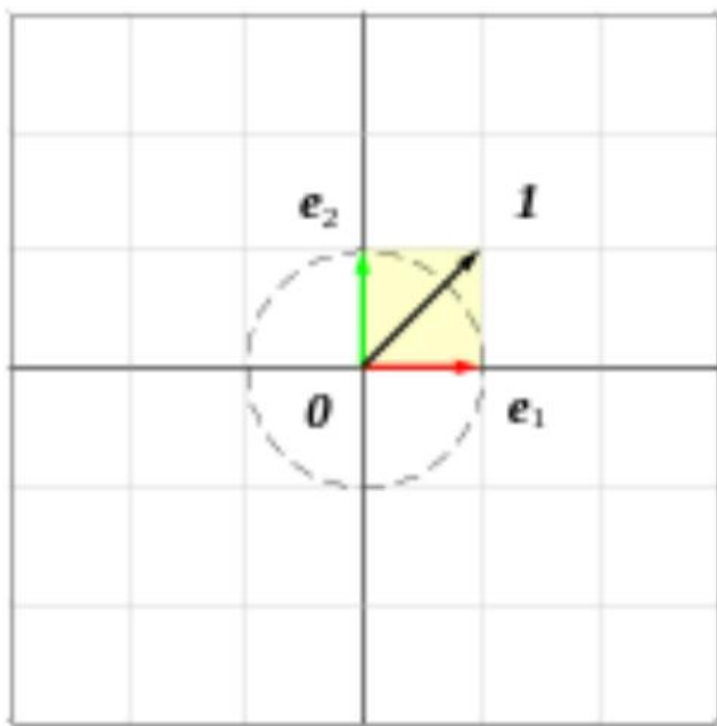


图7 · 矩阵 A 为 2×2 ，水平剪切

- a_1 为水平方向单位向量；单位圆变成了椭圆——这是本书后文LDL分解要讨论的话题



$$A = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$$

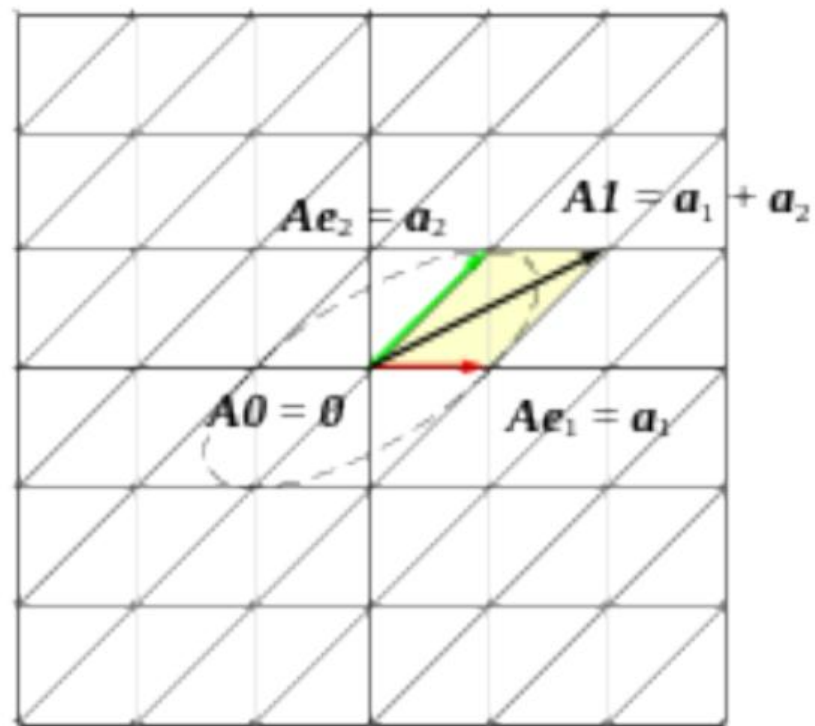


图8 • 矩阵 A 为 2×2 ，正交投影

- $A\mathbf{1} = \mathbf{a}_1 + \mathbf{a}_2 = \mathbf{0}$ ，列向量线性相关，不能构成基底；行列式为0，正交投影矩阵不可逆

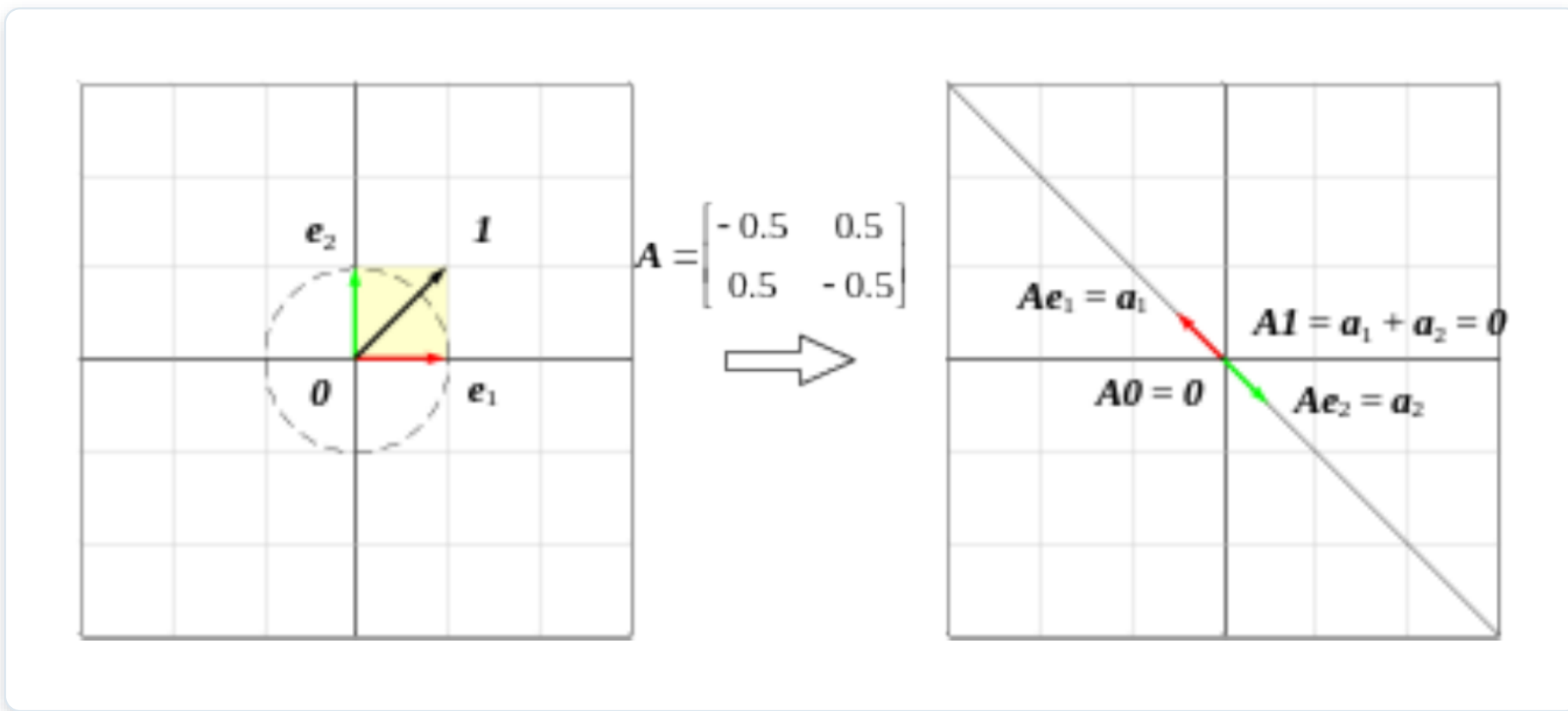
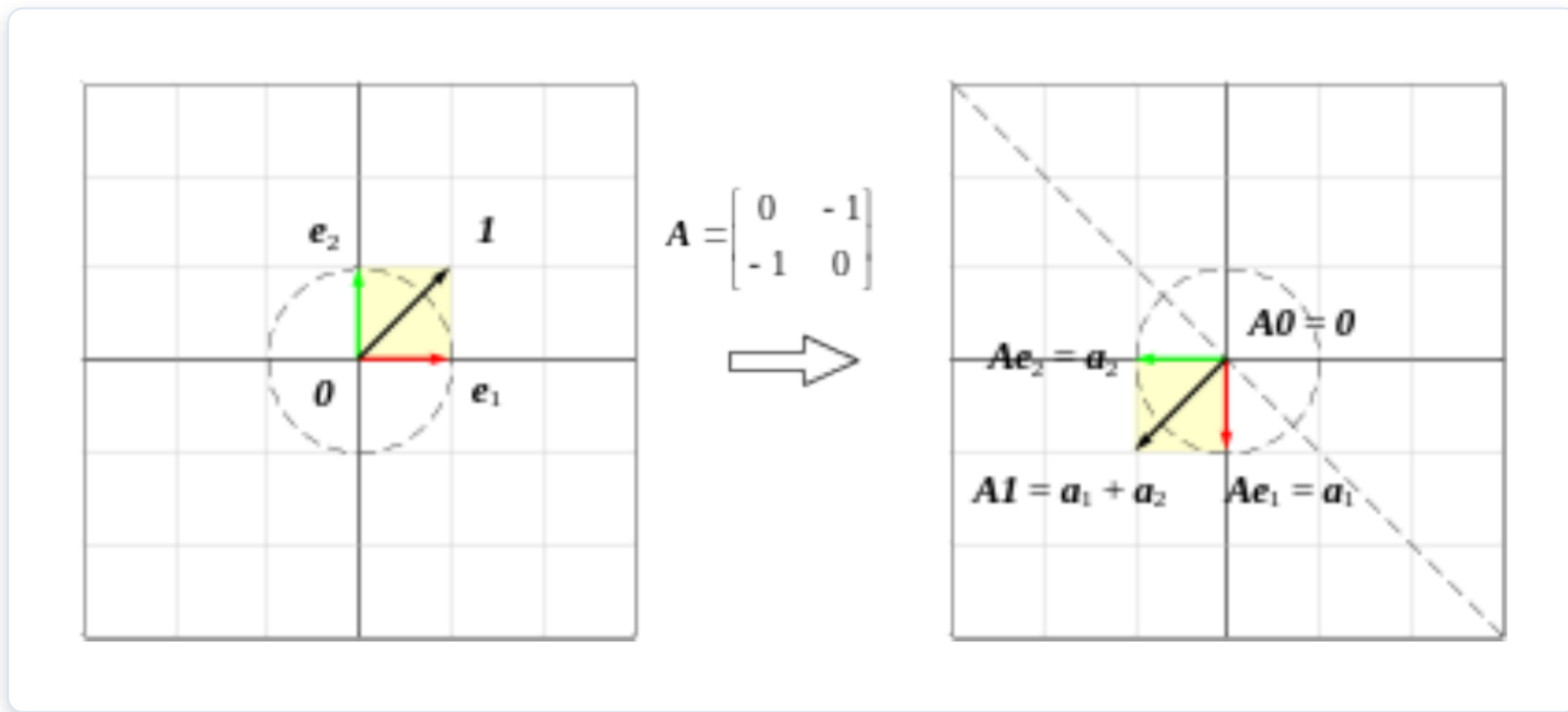


图9 · 矩阵 A 为 2×2 ，镜像

- a_1 、 a_2 都是单位向量且正交，但这个正交矩阵不同于图6中的旋转矩阵——行列式为 -1



自定义函数：网格数据

`grid(A)`: 对整数网格上的每个点做线性变换 A

- 用`numpy.arange()`创建横轴、纵轴的整数网格点
- 用`numpy.meshgrid()`把两个一维数组扩展为二维网格：`xx1`为所有横坐标，`xx2`为所有纵坐标
- 用`numpy.stack()`把两个网格叠加成三维数组，方便后续矩阵操作
- `@`是矩阵乘法符号，`A.T`表示 A 的转置，用来匹配网格数据的维度

```
## 初始化
import numpy as np
import matplotlib.pyplot as plt

## 自定义函数，网格数据
def grid(A):

    x1_grids = np.arange(-10,10+1) # 横轴网格点
    x2_grids = np.arange(-10,10+1) # 纵轴网格点
    xx1, xx2 = np.meshgrid(x1_grids, x2_grids) # 生成网格
    X_grid = np.stack([xx1, xx2], axis=2) # 将网格堆叠为3维数组
    A_X_grid = X_grid @ A.T # A线性变换

    return A_X_grid
```

自定义函数：单位圆数据

`circle(A)`: 生成单位圆数据并做线性变换

- 用`numpy.linspace()`创建0到 2π 之间721个角度点
- 默认半径为1，计算每个角度对应的余弦（横坐标）、正弦（纵坐标）
- 用`numpy.column_stack()`把横纵坐标合并成单位圆上各点的二维数组
- 对单位圆每个点做线性变换，得到变换后的轮廓——本例结果是旋转椭圆

```
## 自定义函数，单位圆数据
def circle(A):
    a
    b    theta_array = np.linspace(0, 2*np.pi, 721)
        # 极坐标到直角坐标
    c    x1_array = np.cos(theta_array)
        x2_array = np.sin(theta_array)
    d    unit_circle = np.column_stack([x1_array, x2_array])
    e
    A_unit_circle = unit_circle @ A.T # A线性变换

    return A_unit_circle
```

自定义函数：单位正方形数据

unit_square(A) : 生成单位正方形四个顶点并做线性变换

- 四个行向量[0,0]、[1,0]、[1,1]、[0,1]代表单位正方形的四个顶点
- 同样用A.T对四个顶点做线性变换，得到变换后的四边形顶点坐标



```
## 自定义函数，单位正方形数据
def unit_square(A):
    unit_square = np.array([[0, 0],
                             [1, 0],
                             [1, 1],
                             [0, 1]])

    A_unit_square = unit_square @ A.T # A线性变换
    return A_unit_square
```

自定义函数：三个特殊列向量

`vectors(A)`：生成 e_1 、 e_2 、全1列向量 $\mathbf{1}$ 的变换结果

- 定义 $\mathbf{e}_1 = [1, 0]^T$ ，用 $A @ \mathbf{e}_1$ 完成线性变换
- 定义 $\mathbf{e}_2 = [0, 1]^T$ ，用 $A @ \mathbf{e}_2$ 完成相同的线性变换
- 将 $\mathbf{e}_1$ 、 $\mathbf{e}_2$ 相加得到全1列向量 $\mathbf{1}$ ，再对其做线性变换



```
## 自定义函数，三个列向量
def vectors(A):
    e1 = np.array([[1],
                   [0]])
    Ae1 = A@e1 # A线性变换

    e2 = np.array([[0],
                   [1]])
    Ae2 = A@e2 # A线性变换

    all_1 = e1 + e2
    A_all_1 = A@all_1 # A线性变换

    return Ae1, Ae2, A_all_1
```

自定义可视化函数

visualize(A): 把网格、单位正方形、单位圆、三个向量画在一张图上

- 调用前面4个自定义函数，分别生成变换后的网格、单位正方形、单位圆、三个向量数据
- 逐行、逐列画出变换后的网格线；用 `quiver()` 绘制三个箭头，`plot()` 绘制变换后单位圆
- 用 `fill()` 绘制变换后单位正方形，背景填充淡黄色，直观呈现平行四边形

```
## 定义可视化函数
def visualize(A):
    fig, ax = plt.subplots()

    # 调用自定义函数创建数据
    A_X_grid = grid(A)
    A_unit_square = unit_square(A)
    A_unit_circle = circle(A)
    Ae1, Ae2, A_all_1 = vectors(A)

    # 绘制变换后的网格
    for i in range(A_X_grid.shape[0]):
        plt.plot(A_X_grid[i, :, 0], A_X_grid[i, :, 1],
                 'k-', linewidth=0.5) # 绘制网格行
    for j in range(A_X_grid.shape[1]):
        plt.plot(A_X_grid[:, j, 0], A_X_grid[:, j, 1],
                 'k-', linewidth=0.5) # 绘制网格列

    # 绘制变换后的列向量
    plt.quiver(0, 0, Ae1[0], Ae1[1],
               angles='xy', scale_units='xy',
               scale=1, color=[1, 0, 0], zorder = 1e5)
    plt.quiver(0, 0, Ae2[0], Ae2[1],
               angles='xy', scale_units='xy',
               scale=1, color=[0, 1, 0], zorder = 1e5)
    plt.quiver(0, 0, A_all_1[0], A_all_1[1],
               angles='xy', scale_units='xy',
               scale=1, color=[0, 0, 0], zorder = 1e5)

    # 绘制变换后单位圆
    plt.plot(A_unit_circle[:,0], A_unit_circle[:,1],
             c = 'k', ls = '--')

    # 绘制变换后单位正方形
    plt.fill(A_unit_square[:,0], A_unit_square[:,1],
             c = '#FFFCC', ls = '--', zorder = 1)

    # 装饰
    plt.axvline(x=0, color='k', zorder=0)
    plt.axhline(y=0, color='k', zorder=0)
    ax.tick_params(axis='both', which='both', length=0,
                  labelbottom=False, labelleft=False)
    ax.set_aspect(1)
    lim = 3; ax.set_xlim([-lim, lim]); ax.set_ylim([-lim, lim])
    plt.xticks(np.arange(-lim, lim))
    plt.yticks(np.arange(-lim, lim))
    ax.grid(linestyle='--', linewidth=0.15, color=[0.8, 0.8, 0.8])
```


3×3方阵

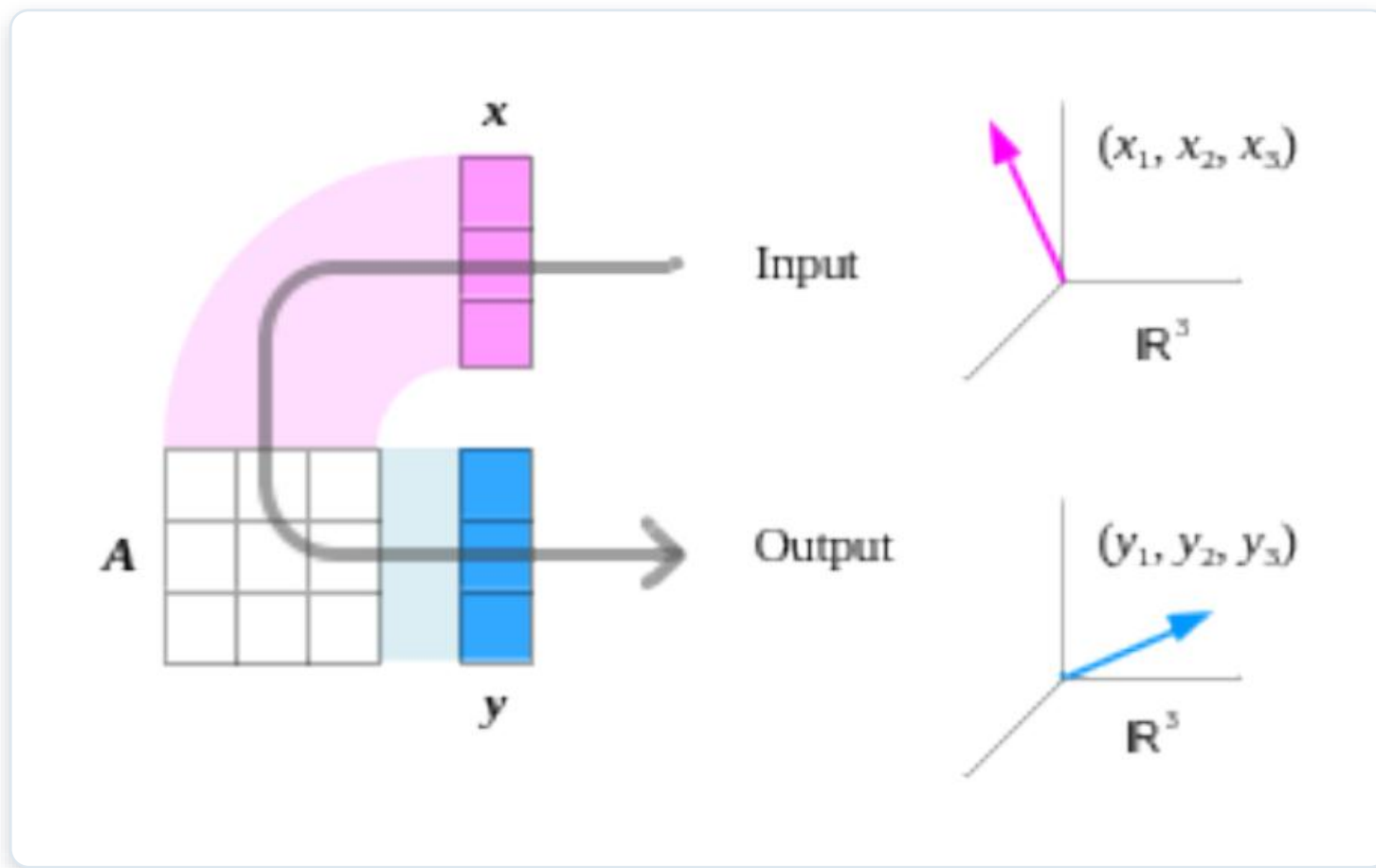
把线性变换扩展到三维空间

$$Ax = y$$

- 3×3矩阵**A**完成3维向量**x**到3维向量**y**的映射；**x**、**y**都是三维空间中的坐标点
- (3×3)@(3×1)中间的3被“消去”，结果形状为(3×1)——与2×2情形完全一致

图10 • 矩阵 A 为 3×3

- 输入 x 、输出 y 都是三维空间中起点位于原点的向量



三维空间的特殊列向量

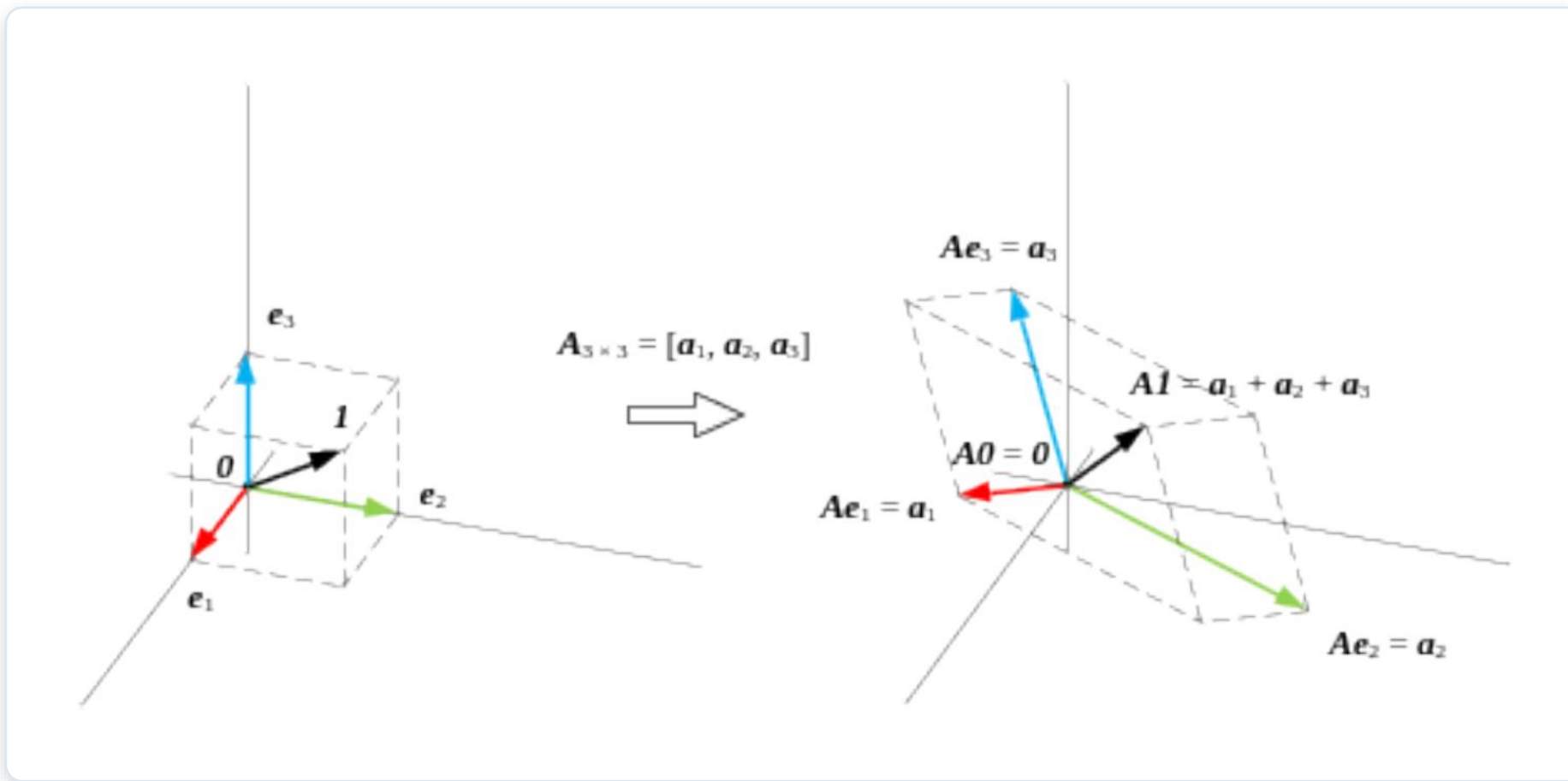
$$Ae_1 = a_1, Ae_2 = a_2, Ae_3 = a_3, A1 = a_1 + a_2 + a_3$$

$$Ae_1 = a_1 \quad Ae_2 = a_2 \quad Ae_3 = a_3 \quad A1 = a_1 + a_2 + a_3$$

- 同样地，三维空间线性变换中原点位置不变； Ae_1 、 Ae_2 、 Ae_3 分别提取 A 的三个列向量
- 单位立方体变成平行六面体：若 a_1 、 a_2 、 a_3 线性无关，网格对应基底 $[a_1, a_2, a_3]$ 张成的空间

图11 • 三维空间中几个特殊列向量

- 在线性变换作用下，单位正方体网格被变换为平行六面体网格：平行且等距，原点位置保持不变



从矩阵乘法看几何变换，观察到了什么？

特殊列向量： $\mathbf{0} \rightarrow \mathbf{0}$ (原点不变)， $\mathbf{e}_1$ 、 $\mathbf{e}_2$ 、 $\mathbf{e}_3 \rightarrow$ 矩阵的列， $\mathbf{1} \rightarrow$ 列向量之和

网格变换：单位正方形/立方体 $\rightarrow$ 平行四边形/六面体，平行且等距

行列式： $|\det(\mathbf{A})|$ 是面积/体积缩放因子；正负号看列向量的相对朝向

五种变换：缩放(对角)、旋转(正交, $\det=1$)、剪切(三角, $\det=1$)、投影(降维, $\det=0$)、镜像(正交, $\det=-1$)

本节关键术语

线性映射 *Linear Mapping* —— 矩阵乘法 $\mathbf{Ax}=\mathbf{y}$ 把输入向量映射为输出向量

旋转 *Rotation* —— 保持长度和角度不变, 正交矩阵, 行列式为1

正交投影 *Orthogonal Projection* —— 垂直投影到子空间, 降维, 行列式为0

正交矩阵 *Orthogonal Matrix* —— 满足 $\mathbf{AA}^T=\mathbf{A}^T\mathbf{A}=\mathbf{I}$ 的方阵

行列式 *Determinant* —— 反映几何变换导致的面积/体积缩放与朝向

缩放 *Scaling* —— 按比例拉伸/压缩, 变换矩阵为对角方阵

剪切 *Shear* —— 沿某方向倾斜变形, 三角矩阵, 行列式为1

镜像 *Reflection* —— 关于过原点直线对称翻转, 正交矩阵, 行列式为-1

基底 *Basis* —— 一组线性无关的向量, 张成整个空间

平行六面体 *Parallelepiped* —— 单位立方体经线性变换后的三维网格单元

延伸阅读

延伸阅读 • 8.2 缩放

本册后续小节 对角矩阵与缩放变换的细节展开

延伸阅读 • 8.3 旋转

本册后续小节 旋转矩阵、旋转角度与正交矩阵

延伸阅读 • 8.5 正交投影

本册后续小节 正交投影、施密特正交化与QR分解

开源仓库

全书代码与图像素材: <https://github.com/Visualize-ML>

习题 Q1 – Q8

Q1 请计算如下4个方阵分别对 $\mathbf{0}$ 、 $\mathbf{e}_1$ 、 $\mathbf{e}_2$ 、 $\mathbf{1}$ 的变换：
(a) $\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ (b) $\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$ (c) $\begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}$ (d) $\begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$ 。

Q3 请分析Q1中哪些矩阵的列向量线性相关？

Q5 请计算Q1每个方阵列向量的长度(L2范数)，以及列向量之间的夹角(用内积)。

Q7 请计算Q1每个方阵的行列式；思考怎么算最便捷，并解释行列式正、负或为0的原因。

Q2 请用语言大致描述Q1每个方阵完成怎样的几何变换；建议先判断，再用本节配套代码逐个可视化检验。

Q4 请分析Q1中哪些矩阵列向量可以构成基底，分析基底类型，并解释原因。

Q6 请在方格纸上绘制Q1每个方阵列向量构成的平行四边形。

Q8 请解释Q1每个方阵是否存在逆矩阵；如果存在，请计算逆矩阵。

习题 Q9 – Q16

Q9 写一个Python函数`transform_square(A)`，接收 2×2 矩阵A，返回单位正方形四个顶点变换后的坐标。

Q11 用`numpy.linalg.det()`验证：旋转矩阵的行列式恒为1，镜像矩阵的行列式恒为-1。

Q13 本节代码用A.T处理网格数据，请解释为什么要用A.T而不是A（提示：网格数据的形状）。

Q15 对 $A = \begin{bmatrix} 3 & 0 \\ 0 & 0.5 \end{bmatrix}$ ，判断它是否为正交矩阵，并计算它对单位圆变换后的图形。

Q10 写一个Python函数`is_orthogonal(A)`，判断方阵A是否满足 $AA^T = A^T A = I$ 。

Q12 构造一个 3×3 置换矩阵，计算它对e1、e2、e3、全1列向量的变换，并说明其几何意义。

Q14 写一个Python函数`grid_area(A)`，用行列式计算矩阵A把单位正方形变换后的平行四边形面积。

Q16 构造一个奇异（不可逆）矩阵，验证它把单位正方形网格压缩成一条直线（降维）。

习题 Q17 – Q25

Q17 用matplotlib分别绘制缩放矩阵 $\text{diag}(2, 0.5)$ 和旋转30度矩阵对单位圆的变换结果，比较区别。

Q19 对 3×3 矩阵 $A = \text{diag}(1, 1, -1)$ ，计算它对 e_1 、 e_2 、 e_3 、全1列向量的变换，并说明这是哪种几何变换。

Q21 自学：用numpy计算一个 2×2 矩阵的特征值、特征向量，并思考与本节缩放、旋转变换的联系。

Q23 写一个Python函数`compose(*matrices)`，返回多个矩阵按顺序连乘的结果，验证 $(AB)^T = B^T A^T$ 。

Q25 综合练习：设计一个 3×3 矩阵，使其同时具有缩放和剪切的效果，并用代码可视化验证。

Q18 请证明：对任意方阵 A ， $A0=0$ 恒成立（提示：利用矩阵乘法的线性性质）。

Q20 写一个Python函数`vectors_3d(A)`，仿照代码4，对 3×3 矩阵 A 生成 e_1 、 e_2 、 e_3 变换后的向量。

Q22 修改代码5的`visualize`函数，让坐标轴范围根据变换结果自动缩放，而不是固定为 $[-3, 3]$ 。

Q24 任选一个 2×2 矩阵 A ，比较 A 与 A^T 对同一个单位正方形网格的变换结果，是否相同？为什么？

10个可以和AI互动的Prompt

- 1 请解释为什么矩阵 A 对零向量 $\mathbf{0}$ 的变换恒为 $\mathbf{0}$ ，这对判断“是否为线性变换”有什么用？
- 2 请证明： $(AB)^T = B^T A^T$ ，并说明这对理解连续几何变换的复合有什么帮助。
- 3 请用Python演示：分别用缩放矩阵、旋转矩阵、剪切矩阵变换同一个单位正方形，比较网格形状的差异。
- 4 请解释为什么正交矩阵一定满足列向量长度为1且两两正交，这与旋转、镜像变换有什么关系？
- 5 请解释行列式的正负号如何反映列向量之间的相对朝向。

10个可以和AI互动的Prompt

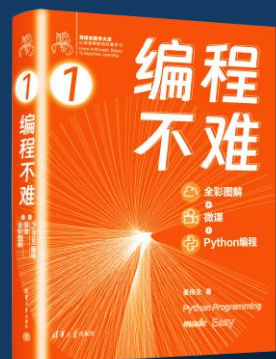
- 6 请解释正交投影为什么会降维，行列式为0在这里意味着什么？
- 7 请用Python验证： 3×3 矩阵 $\mathbf{A}$ 对单位立方体的变换结果是平行六面体，其体积等于 $|\det(\mathbf{A})|$ 。
- 8 请解释矩阵 $\mathbf{A}$ 的列向量是否线性无关，与它是否能构成基底之间的关系。
- 9 请设计一个实验，比较矩阵 $\mathbf{A}$ 与其转置 $\mathbf{A}^T$ 对同一个单位正方形网格的变换结果有何不同。
- 10 请解释为什么本章说“几何变换是本书后续矩阵分解的基础”，试举一个矩阵分解的例子说明。

数学

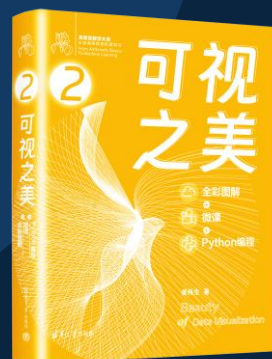
数学基础

线性代数

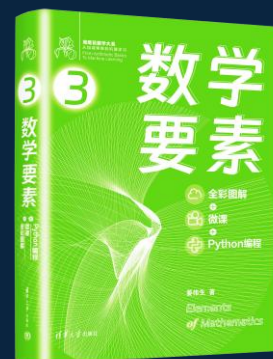
概率统计



Python编程



数据可视化



工具

数据分析
图和网络

回归、降维
分类、聚类

实践

